

Data Structures and Algorithms

Exercise 1: Inventory Management System

Solution:

```
using System;
```

```
using System.Collections.Generic;
```

```
class Product
```

```
{
```

```
    public int ProductId;
```

```
    public string ProductName;
```

```
    public int Quantity;
```

```
    public double Price;
```

```
    public Product(int id, string name, int qty, double price)
```

```
    {
```

```
        ProductId = id;
```

```
        ProductName = name;
```

```
        Quantity = qty;
```

```
        Price = price;
```

```
    }
```

```
}
```

```
class Program
```

```
{
```

```
    static Dictionary<int, Product> inventory = new Dictionary<int, Product>();
```

```
    static void AddProduct()
```

```
{  
    Console.Write("Product ID: ");  
    int id = int.Parse(Console.ReadLine());  
  
    Console.Write("Product Name: ");  
    string name = Console.ReadLine();  
  
    Console.Write("Quantity: ");  
    int qty = int.Parse(Console.ReadLine());  
  
    Console.Write("Price: ");  
    double price = double.Parse(Console.ReadLine());  
  
    inventory[id] = new Product(id, name, qty, price);  
  
    Console.WriteLine("Product Added Successfully.");  
}
```

```
static void UpdateProduct()  
{  
    Console.Write("Enter Product ID: ");  
    int id = int.Parse(Console.ReadLine());  
  
    if (inventory.ContainsKey(id))  
    {  
        Console.Write("New Quantity: ");  
        inventory[id].Quantity = int.Parse(Console.ReadLine());  
  
        Console.Write("New Price: ");
```

```

        inventory[id].Price = double.Parse(Console.ReadLine());

        Console.WriteLine("Product Updated.");
    }
    else
    {
        Console.WriteLine("Product Not Found.");
    }
}

static void DeleteProduct()
{
    Console.Write("Enter Product ID: ");
    int id = int.Parse(Console.ReadLine());

    if (inventory.Remove(id))
        Console.WriteLine("Product Deleted.");
    else
        Console.WriteLine("Product Not Found.");
}

static void DisplayProducts()
{
    foreach (var p in inventory.Values)
    {
        Console.WriteLine($"{p.ProductId} {p.ProductName} {p.Quantity} {p.Price}");
    }
}

```

```

static void Main()
{
    while (true)
    {
        Console.WriteLine("\n1.Add 2.Update 3.Delete 4.Display 5.Exit");
        int choice = int.Parse(Console.ReadLine());

        switch (choice)
        {
            case 1: AddProduct(); break;
            case 2: UpdateProduct(); break;
            case 3: DeleteProduct(); break;
            case 4: DisplayProducts(); break;
            case 5: return;
        }
    }
}

```

Exercise 2: E-commerce Platform Search Function

Solution:

```
using System;
```

```
class Product
```

```

{
    public int ProductId;
    public string ProductName;
    public string Category;

```

```

public Product(int id, string name, string category)
{
    ProductId = id;
    ProductName = name;
    Category = category;
}
}

class Program
{
    static int LinearSearch(Product[] products, string key)
    {
        for (int i = 0; i < products.Length; i++)
        {
            if (products[i].ProductName == key)
                return i;
        }
        return -1;
    }

    static int BinarySearch(Product[] products, string key)
    {
        int left = 0, right = products.Length - 1;

        while (left <= right)
        {
            int mid = (left + right) / 2;

```

```

        int cmp = string.Compare(products[mid].ProductName, key);

        if (cmp == 0)
            return mid;
        else if (cmp < 0)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1;
}

static void Main()
{
    Product[] products =
    {
        new Product(1, "Apple", "Fruit"),
        new Product(2, "Banana", "Fruit"),
        new Product(3, "Laptop", "Electronics"),
        new Product(4, "Phone", "Electronics")
    };

    Console.Write("Enter Product Name: ");
    string key = Console.ReadLine();

    int index = LinearSearch(products, key);

    if (index != -1)

```

```

        Console.WriteLine("Found using Linear Search");
    else
        Console.WriteLine("Not Found");

    Array.Sort(products, (a, b) => a.ProductName.CompareTo(b.ProductName));

    index = BinarySearch(products, key);

    if (index != -1)
        Console.WriteLine("Found using Binary Search");
    else
        Console.WriteLine("Not Found");
}
}

```

Exercise 3: Sorting Customer Orders

Solution:

```

using System;

class Order
{
    public int OrderId;
    public string CustomerName;
    public double TotalPrice;

    public Order(int id, string name, double price)
    {
        OrderId = id;
    }
}

```

```

        CustomerName = name;

        TotalPrice = price;
    }
}

class Program
{
    static void BubbleSort(Order[] orders)
    {
        int n = orders.Length;

        for (int i = 0; i < n - 1; i++)
        {
            for (int j = 0; j < n - i - 1; j++)
            {
                if (orders[j].TotalPrice > orders[j + 1].TotalPrice)
                {
                    Order temp = orders[j];
                    orders[j] = orders[j + 1];
                    orders[j + 1] = temp;
                }
            }
        }
    }

    static int Partition(Order[] arr, int low, int high)
    {
        double pivot = arr[high].TotalPrice;
        int i = low - 1;

```



```

    for (int j = low; j < high; j++)
    {
        if (arr[j].TotalPrice < pivot)
        {
            i++;
            Order temp = arr[i];
            arr[i] = arr[j];
            arr[j] = temp;
        }
    }

    Order t = arr[i + 1];
    arr[i + 1] = arr[high];
    arr[high] = t;

    return i + 1;
}

static void QuickSort(Order[] arr, int low, int high)
{
    if (low < high)
    {
        int pi = Partition(arr, low, high);

        QuickSort(arr, low, pi - 1);
        QuickSort(arr, pi + 1, high);
    }
}

```

```

static void Display(Order[] orders)
{
    foreach (var o in orders)
    {
        Console.WriteLine($"{o.OrderId} {o.CustomerName} {o.TotalPrice}");
    }
}

static void Main()
{
    Order[] orders =
    {
        new Order(1,"John",5000),
        new Order(2,"Alex",2000),
        new Order(3,"David",8000),
        new Order(4,"Mike",3000)
    };

    Console.WriteLine("Bubble Sort:");
    BubbleSort(orders);
    Display(orders);

    Console.WriteLine("\nQuick Sort:");
    QuickSort(orders, 0, orders.Length - 1);
    Display(orders);
}
}

```

Exercise 4: Employee Management System

Solution:

using System;

class Employee

{

public int EmployeeId;

public string Name;

public string Position;

public double Salary;

public Employee(int id, string name, string position, double salary)

{

EmployeeId = id;

Name = name;

Position = position;

Salary = salary;

}

}

class Program

{

static Employee[] employees = new Employee[100];

static int count = 0;

static void AddEmployee()

```

{
    Console.Write("ID: ");
    int id = int.Parse(Console.ReadLine());

    Console.Write("Name: ");
    string name = Console.ReadLine();

    Console.Write("Position: ");
    string position = Console.ReadLine();

    Console.Write("Salary: ");
    double salary = double.Parse(Console.ReadLine());

    employees[count++] = new Employee(id, name, position, salary);
}

```

```

static void SearchEmployee()
{
    Console.Write("Enter ID: ");
    int id = int.Parse(Console.ReadLine());

    for (int i = 0; i < count; i++)
    {
        if (employees[i].EmployeeId == id)
        {
            Console.WriteLine(employees[i].Name);
            return;
        }
    }
}

```

```

        Console.WriteLine("Not Found");
    }

    static void Traverse()
    {
        for (int i = 0; i < count; i++)
        {
            Console.WriteLine($"{employees[i].EmployeeId} {employees[i].Name}");
        }
    }

    static void DeleteEmployee()
    {
        Console.Write("Enter ID: ");
        int id = int.Parse(Console.ReadLine());

        for (int i = 0; i < count; i++)
        {
            if (employees[i].EmployeeId == id)
            {
                for (int j = i; j < count - 1; j++)
                    employees[j] = employees[j + 1];

                count--;
                Console.WriteLine("Deleted");
                return;
            }
        }
    }

```

```

    }

    static void Main()
    {
        while (true)
        {
            Console.WriteLine("\n1.Add 2.Search 3.Display 4.Delete 5.Exit");
            int ch = int.Parse(Console.ReadLine());

            switch (ch)
            {
                case 1: AddEmployee(); break;
                case 2: SearchEmployee(); break;
                case 3: Traverse(); break;
                case 4: DeleteEmployee(); break;
                case 5: return;
            }
        }
    }
}

```

Exercise 5: Task Management System

Solution:

```
using System;
```

```
class TaskNode
```

```

{
    public int TaskId;

```

```

public string TaskName;

public string Status;

public TaskNode Next;


public TaskNode(int id, string name, string status)
{
    TaskId = id;
    TaskName = name;
    Status = status;
}
}


class Program
{
    static TaskNode head = null;


    static void AddTask(int id, string name, string status)
    {
        TaskNode newNode = new TaskNode(id, name, status);

        if (head == null)
        {
            head = newNode;
            return;
        }

        TaskNode temp = head;

        while (temp.Next != null)

```

```

        temp = temp.Next;

        temp.Next = newNode;
    }

    static void Traverse()
    {
        TaskNode temp = head;

        while (temp != null)
        {
            Console.WriteLine($"{temp.TaskId} {temp.TaskName} {temp.Status}");
            temp = temp.Next;
        }
    }

    static void Main()
    {
        AddTask(1, "Coding", "Pending");
        AddTask(2, "Testing", "Completed");

        Traverse();
    }
}

```


Exercise 6: Library Management System

Solution:

using System;

class Book

{

 public int BookId;

 public string Title;

 public string Author;

 public Book(int id, string title, string author)

 {

 BookId = id;

 Title = title;

 Author = author;

 }

}

class Program

{

 static Book LinearSearch(Book[] books, string title)

 {

 foreach (Book book in books)

 {

 if (book.Title == title)

 return book;

 }

 return null;

```
}

static void Main()
{
    Book[] books =
    {
        new Book(1,"Java","James"),
        new Book(2,"CSharp","Microsoft"),
        new Book(3,"Python","Guido")
    };

    Console.Write("Enter Book Title: ");
    string title = Console.ReadLine();

    Book result = LinearSearch(books, title);

    if (result != null)
        Console.WriteLine("Found: " + result.Author);
    else
        Console.WriteLine("Not Found");
    }
}
```

Exercise 7: Financial Forecasting

Solution:

using System;

class Program

{

static double FutureValue(double currentValue, double growthRate, int years)

{

if (years == 0)

return currentValue;

return FutureValue(

currentValue * (1 + growthRate),

growthRate,

years - 1

);

}

static void Main()

{

Console.Write("Current Value: ");

double current = double.Parse(Console.ReadLine());

Console.Write("Growth Rate (%): ");

double rate = double.Parse(Console.ReadLine()) / 100;

```
Console.Write("Years: ");  
int years = int.Parse(Console.ReadLine());  
  
double result = FutureValue(current, rate, years);  
  
Console.WriteLine("Future Value = " + result);  
}  
}
```